SERVICIO NACIONAL DE APRENDIZAJE — SENA

HOTEL GRAN MARQUES

Sistema de Gestión Hotelera

Evidencia GA7-220501096-AA5-EV03

Diseño y desarrollo de servicios web – proyecto

APRENDIZ
DANIEL ALEJANDRO RAMIREZ HERRERA
IDENTIFICACIÓN
C.C. 1036966567
PROGRAMA DE FORMACIÓN
Tecnología en Análisis y Desarrollo de Software (ADSO)
FICHA
3336187
FECHA
18 de octubre de 2026

Documentación de API REST, catálogo de servicios, contratos JSON, validaciones y versionamiento.

Contenido

- 1. Introducción
- 2. Objetivo y alcance
- 3. Arquitectura de la API
- 4. Catálogo y documentación de servicios
- 5. Modelos de solicitud y reglas de negocio
- 6. Seguridad y manejo de errores
- 7. Pruebas automatizadas y Postman
- 8. Versionamiento con Git
- 9. Limitaciones y mejoras futuras
- Referencias

Este informe complementa los archivos ejecutables de la carpeta comprimida. La definición completa de cada endpoint se encuentra en docs/API_REFERENCE.md y en docs/openapi.json.

1. Introducción

Esta evidencia amplía el sistema web Hotel Gran Marques mediante servicios web REST que exponen las funciones principales de la gestión hotelera. Los servicios reciben solicitudes HTTP, validan los datos, ejecutan reglas del negocio, guardan la información y devuelven respuestas JSON con códigos HTTP que indican el resultado de cada operación.

La API conserva el frontend desarrollado previamente y agrega una capa de servicios para autenticación, huéspedes, habitaciones, reservas, servicios adicionales, facturación y pagos, mantenimiento y reportes. La documentación permite que otro desarrollador o cliente de prueba consuma los endpoints sin tener que revisar cada línea de código.

2. Objetivo y alcance

Objetivo general: diseñar, codificar y documentar los servicios web necesarios para soportar las operaciones centrales del Sistema de Gestión Hotelera Hotel Gran Marques, dejando el proyecto organizado para su versionamiento mediante Git.

2.1 Alcance funcional

- Registro de usuarios y autenticación con token Bearer.
- Consulta y administración de huéspedes.
- Consulta y administración del catálogo de habitaciones y servicios.
- Creación, búsqueda, actualización y control de reservas.
- Asociación de consumos a reservas.
- Generación de facturas y registro de pagos parciales.
- Creación y seguimiento de solicitudes de mantenimiento.
- Consulta de indicadores operativos y gerenciales.

2.2 Tecnologías

Componente	Tecnología / archivo	Responsabilidad
Servidor	Node.js y módulo nativo http	Recibir solicitudes y entregar respuestas JSON.
Autenticación	crypto / JWT HS256	Hash de contraseñas, firma y validación de token.
Persistencia académica	JSON local en api/data	Conservar usuarios y recursos del hotel entre ejecuciones.
Contrato de API	docs/openapi.json	Describir rutas para herramientas compatibles con OpenAPI.
Pruebas	Postman y scripts/api-test.js	Verificar respuestas, reglas de negocio y códigos HTTP.

3. Arquitectura de la API

El cliente web o Postman envía solicitudes al servidor Node.js. La ruta y el método identifican el servicio; las operaciones de negocio validan el token Bearer y los datos recibidos. La información se almacena en archivos JSON locales. La capa de autenticación mantiene un archivo de usuarios separado de los datos operativos del hotel.

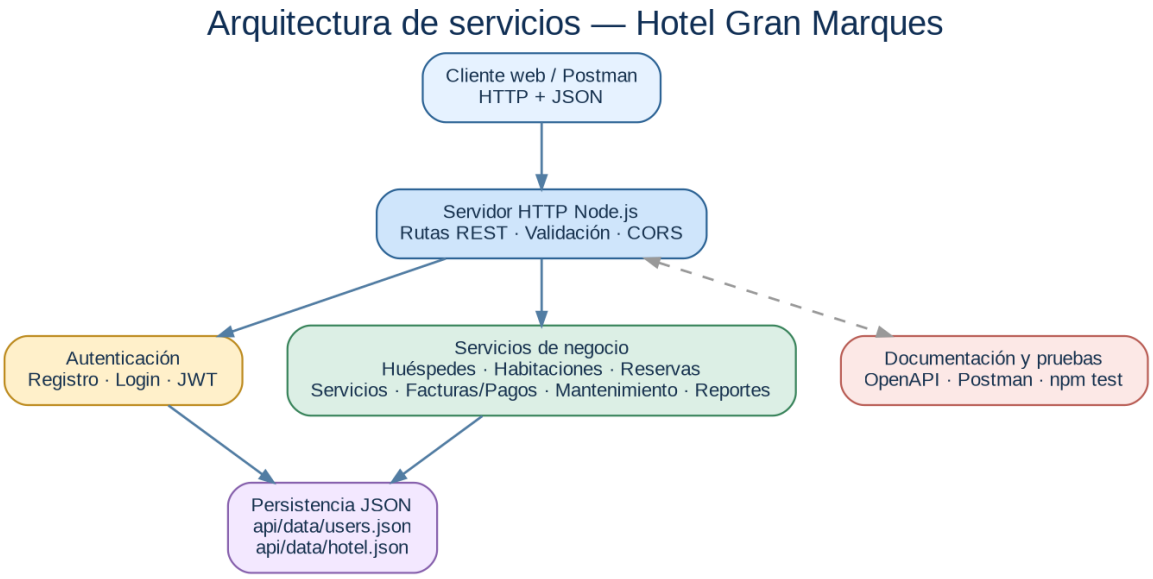


Figura 1. Arquitectura lógica de los servicios web del Hotel Gran Marques.

3.1 Convención de respuesta

```
{
  "success": true,
  "message": "Operación realizada correctamente.",
  "data": { }
}
```

Los errores contienen success=false y un mensaje explicativo. Se utilizan, entre otros, 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found y 409 Conflict.

4. Catálogo y documentación de servicios

Los siguientes cuadros resumen los endpoints de la versión 2.0.0. Las rutas se consumen desde la base URL http://localhost:3000 durante el desarrollo local. Excepto salud, información básica y autenticación pública, las rutas de negocio requieren el encabezado Authorization: Bearer <token>.

4.1 Sistema y autenticación

Método	Endpoint	Propósito / permiso
GET	/api/health	Estado y versión del servicio; público.
GET	/api	Información general y ubicación de documentación; público.
POST	/api/auth/register	Registrar usuario; público. Crea rol Recepción.
POST	/api/auth/login	Validar credenciales y emitir token; público.
GET	/api/auth/me	Consultar perfil del usuario autenticado; Bearer.

4.2 Servicios de negocio

Recurso	Endpoints principales	Reglas relevantes
Huéspedes	GET/POST /api/guests; GET/PATCH/PUT/DELETE /api/guests/{id}	Documento único; nombre y documento obligatorios; eliminación administrativa sin reservas.
Habitaciones	GET/POST /api/rooms;	Número único; tarifa no negativa; cambios del catálogo solo

Recurso	Endpoints principales	Reglas relevantes
	GET/PATCH/PUT/DELETE /api/rooms/{id}	administración.
Reservas	GET/POST /api/reservations; GET/PATCH/PUT/DELETE /api/reservations/{id}	Huésped y habitación existentes, rango de fechas válido, capacidad y prevención de cruces.
Consumos	POST /api/reservations/{id}/services	Vincula un servicio activo y cantidad a una reserva vigente.
Servicios	GET/POST /api/services; GET/PATCH/PUT/DELETE /api/services/{id}	Precio no negativo; escrituras administrativas; no borrar servicios ya consumidos.
Facturas	GET/POST /api/invoices; GET /api/invoices/{id}	Una factura por reserva en esta versión; total calculado desde alojamiento y consumos.
Pagos	POST /api/invoices/{id}/payments	Monto positivo; no puede exceder el saldo; método controlado.
Mantenimiento	GET/POST /api/maintenance; GET/PATCH/PUT /api/maintenance/{id}	Prioridad y estado dentro de valores admitidos; habitación opcional.
Reportes	GET /api/reports/summary	Agrega ocupación, reservas, facturas, recaudo y mantenimiento.

El listado completo, incluidos los filtros de consulta, se encuentra en docs/ENDPOINTS_API.txt. Los cuerpos JSON y respuestas detalladas se encuentran en docs/API_REFERENCE.md.

5. Modelos de solicitud y reglas de negocio

5.1 Registro e inicio de sesión

```
POST /api/auth/register
{
  "name": "Andrea Demo",
  "username": "andrea_demo",
  "email": "andrea@example.test",
  "password": "ClaveSegura2026!"
}

POST /api/auth/login
{ "username": "admin", "password": "123456" }
```

El registro exige nombre, usuario, correo válido y contraseña entre 8 y 72 caracteres. Las claves se guardan con scrypt y sal aleatoria. El login devuelve un token firmado, su vigencia y el perfil público; nunca devuelve el hash.

5.2 Crear una reserva

```
POST /api/reservations
{
  "guestId": "UUID_DEL_HUESPED",
  "roomId": "UUID_DE_HABITACION",
  "checkIn": "2026-12-10",
  "checkOut": "2026-12-12",
  "guestsCount": 2,
  "notes": "Llegada en la tarde"
}
```

Antes de guardar la reserva, el servicio verifica que existan huésped y habitación, que la habitación esté disponible, que el rango sea válido, que la capacidad alcance para todos los huéspedes y que no se superponga con otra reserva activa para la misma habitación. Si detecta conflicto devuelve 409.

5.3 Facturación y pagos

```
POST /api/invoices
{ "reservationId": "UUID_DE_RESERVA" }

POST /api/invoices/UUID_DE_FACTURA/payments
{ "amount": 10000, "method": "Transferencia", "reference": "TRX-2026-001" }
```

La factura se calcula sumando noches por tarifa de habitación y los consumos asociados. El entorno académico usa impuesto cero, pendiente de parametrización tributaria del negocio. El pago no puede superar el saldo; la factura queda Pendiente, Parcial o Pagada según el monto recibido.

6. Seguridad y manejo de errores

- Las contraseñas se almacenan mediante scrypt con sal aleatoria; no se guardan en texto plano.
- Los tokens Bearer se firman con HMAC-SHA256 y expiran después de una hora.

- Las rutas operativas verifican token; las escrituras de habitaciones y catálogo de servicios requieren rol de administración.
- Las entradas se validan antes de guardar; se controlan formatos, duplicados, fechas, capacidad, saldos y relaciones.
- Los errores usan códigos HTTP apropiados y mensajes sin revelar hashes o credenciales.
- JWT_SECRET debe configurarse en .env fuera del repositorio; .env y archivos de datos se excluyen en .gitignore.

Nota: el almacenamiento en JSON y las cuentas de demostración son solo para aprendizaje local. Antes de producción se requiere base de datos transaccional, HTTPS, secreto robusto, limitación de intentos, auditoría, copias de seguridad y configuración CORS restrictiva.

7. Pruebas automatizadas y Postman

El archivo scripts/api-test.js ejecuta solicitudes reales contra la API local. En la ejecución de referencia incluida en el paquete se aprobaron 36 de 36 verificaciones y no se registraron fallos. Las pruebas cubren salud, autenticación, perfil, registro, credenciales incorrectas, creación y búsqueda de huésped, habitaciones, reservas, prevención de cruces, servicios, generación de factura, duplicados, abonos, mantenimiento, reportes, permisos de rol y acceso sin token.

Herramienta / archivo	Uso
npm test	Ejecuta las pruebas de integración contra http://localhost:3000.
docs/resultado_pruebas_api.json	Resultados estructurados generados en la última ejecución.
postman/Hotel_Gran_Marques_API_EV03.postman_collection.json	Colección de 22 solicitudes con assertions y variables.
postman/Hotel_Gran_Marques_Local_EV03.postman_environment.json	Ambiente local con baseUrl.
docs/openapi.json	Definición OpenAPI importable en herramientas compatibles.

8. Versionamiento con Git

La evidencia debe integrarse al repositorio existente del proyecto. El archivo VERSIONAMIENTO_GIT.md contiene los comandos de referencia y pasos para verificar el remoto. El ZIP no incluye la carpeta interna .git; el historial se conserva en el repositorio local de la aprendiz.

```
git status
git add .
git commit -m "GA7-AA5-EV03: desarrollar servicios REST para Hotel Gran Marques"
git push origin main
git remote -v
```

Después de publicar los cambios, se debe copiar la URL real del repositorio en ENLACE_REPOSITORIO.txt. El enlace queda pendiente hasta que se publique en la cuenta de Git elegida.

9. Limitaciones y mejoras futuras

- Migrar la persistencia de JSON a PostgreSQL, MySQL u otra base de datos transaccional.
- Agregar filtros, paginación y ordenamiento para catálogos extensos.
- Definir políticas de disponibilidad por fecha y estados operativos de check-in/check-out según los procedimientos reales del hotel.
- Parametrizar impuestos y numeración fiscal antes de emitir comprobantes reales.
- Añadir rate limiting, auditoría, rotación de secretos, HTTPS, pruebas de concurrencia y autorización por permisos más específicos.
- Conectar progresivamente todos los formularios del frontend a los endpoints de negocio; el frontend actual ya usa la API para autenticación, mientras los demás módulos conservan parte de su estado de demostración en el navegador.

Referencias

Node.js. (s. f.). HTTP. Node.js Documentation. https://nodejs.org/api/http.html

Node.js. (s. f.). Crypto. Node.js Documentation. https://nodejs.org/api/crypto.html

OpenAPI Initiative. (s. f.). OpenAPI Specification. https://spec.openapis.org/oas/latest.html

Postman. (s. f.). Write scripts to test API response data. Postman Learning Center. https://learning.postman.com/docs/tests-and-scripts/write-scripts/test-scripts/

OWASP Foundation. (s. f.). Password Storage Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Servicio Nacional de Aprendizaje [SENA]. (2026). Evidencia GA7-220501096-AA5-EV03: Diseño y desarrollo de servicios web – proyecto. Guía de aprendizaje.